

WorldForge / 世界观工坊 —— 插件开发指南

项	内容
版本	v0.1（宿主 <code>api.version = "0.1.0"</code> ）
日期	2026-02-14
适用对象	想扩展卡片类型、面板、命令、主题、导出器的进阶用户与开发者
相关文档	<code>docs/需求规格说明书-v0.1.md</code> （FR-14）、 <code>docs/架构设计.md</code> （§5.8）、 <code>docs/数据模型.md</code> （ <code>plugins</code> 表）

⚠ 实现状态声明（请先读这一段）

本文档最初是作为插件 API 的**设计稿**写的，其中一部分接口**并未实现**。
为避免照着写却跑不起来，这里明确区分「已实现」与「仍是设计」。

已实现—— `PluginAPI` 的全部成员就是插件能用的全部能力：

```
// worldforge:plugin-api version, pluginId, settings, registerCardType, registerCardField, registerPanel, registerCommand, registerCardAction
```

这一行由 `scripts/selftest-specs.mjs` 与 `src/types/plugin-api.ts` 强制比对，
源码增删成员而文档没跟上时 `npm test` 会失败。

尚未实现（下文相关段落请勿照抄）：

`api.cards.*`、`api.tags.*`、`api.relations.*`、`api.ui.openCard`、`api.worlds.*`、
`api.declareSettings`、`api.pluginSettings`、`api.storage.remove/keys`、
导出 `deactivate()`、以及「连续 3 次异常自动禁用」。
对应的正确用法：只读查询用 `api.query.*`；设置声明在源码里导出 `manifest.settings`
（见 §4.3）；插件清理用 `activate` 的**返回值**（宿主不会调用 `deactivate`）。

第 3.1~3.3 节的示例与 §4.1 的接口表**尚未按实现重写**，目前只能当设计参考；
可直接运行、且与实现一致的范本是 `src/lib/plugin/builtins.ts` 里的 4 个内置插件，
以及 §3.4 的「图库灯箱」（源码就在仓库的 `plugins/gallery-lightbox.js`）。

1 插件是什么

1.1 定义

插件是一个**单文件 ES 模块**，导出 `activate(api)` 函数（或 `export default { activate }`）。宿主把它当作文本存进 `plugins.code`，启用时用 `Blob` + 动态 `import()` 载入，调用 `activate` 并注入宿主 API。

```
// 最小可用插件：只打印一行版本信息
export function activate(api) {
  api.toast('插件已启用，宿主版本 ' + api.version);
}
```

能力	说明
扩展卡片类型	<code>registerCardType</code> 新增类型， <code>registerCardField</code> 为已有类型追加字段
扩展界面	<code>registerPanel</code> 注册侧边面板（时间轴、关系图谱、统计...）
扩展操作	<code>registerCommand</code> 注册命令面板指令（ <code>Ctrl/Cmd + K</code> 可搜到）
扩展外观	<code>registerTheme</code> 注册配色主题
扩展导出	<code>registerExporter</code> 注册导出格式（Markdown / CSV / 自定义标记语言...）
监听数据变化	<code>on('card:save' / 'card:delete' / 'app:ready')</code>

能力	说明
只读查询	<code>api.cards / api.tags / api.relations</code>
私有存储	<code>api.storage</code> （按插件 id 隔离的键值存储）

1.2 不能做什么（v0.1 边界）

限制	说明
不能写数据库	<code>api.cards/tags/relations</code> 只读；要持久化自定义数据请用 <code>api.storage</code> 或自定义字段
不能改宿主 UI 结构	只能新增面板 / 命令 / 类型 / 主题 / 导出器，不能替换侧栏、路由或快捷键表。想作用于 已有 界面（例如给图库加放大预览），用事件委托挂在宿主标记过的 DOM 上，见 §3.4
不能联网下载依赖	插件是单文件，无 <code>import</code> 外部包能力（可用宿主已提供的全局对象与标准 Web API）
不能拦截或取消写操作	事件为通知型，不支持 <code>preventDefault</code>
界面样式受宿主限制	插件里的 Tailwind 类名要参与扫描才会生成 CSS： <code>tailwind.config.js</code> 已包含 <code>src/</code> 与 <code>plugins/</code> 。粘贴进来的代码不在扫描范围内，请用内联 <code>style</code> 或宿主已用到的类名
不支持沙箱	见 §1.3
不支持插件间依赖	不解析依赖关系与版本约束（v0.4 之后考虑）

1.3 信任模型与风险提示

v0.1 无沙箱。 插件与应用同源、同权限运行，处于同一个 JavaScript 上下文，因此：

- 可以读取你世界观的**全部数据**（通过 `api` 或直接访问页面内对象）；
- 可以发起网络请求（把数据发送到外部服务器在技术上可行）；
- 可以修改页面 DOM 与本地存储；
- 一个未捕获的异常可能影响界面（宿主会 `try/catch` 载入与事件回调，并把出错的插件置为停用，但不能保证 100% 不受影响）。

| 角色 | 责任 |

| --- | --- |

| 宿主 | 安装前展示风险提示并要求勾选确认；载入与 `activate` 抛错会被 `try/catch` 捕获、`toast` 提示并把该插件置为停用；已注册内容的注销由宿主统一负责 |

| 用户 | **只安装自己信任来源的插件**；安装前阅读代码（单文件，可直接通读）；不确定时先在测试世界观上启用 |

| 插件作者 | 只做声明过的功能；不发网络请求；不读取无关数据；异常自行捕获或用 `api.toast` 反馈 |

卸载 / 禁用时宿主会：调用 `activate` 返回的清理函数 → 注销该插件的全部注册项 → 取消其事件订阅 → `revokeObjectURL` 释放 Blob URL。**宿主不会调用 `deactivate()`**（该导出不存在），需要清理定时器 / 监听器时请从 `activate` 里 `return` 一个函数。插件的设置值随记录一起保存在 `plugins.settings`，卸载插件时该行一并删除。

2 插件包结构与清单

单文件即插件包。文件顶部用注释写清单（宿主不会解析这段注释，它是给人看的；真实元信息在安装时由界面填写并写入 `plugins` 表）。

```
/**
 * @plugin world-annals
 * @name 世界年鉴
 * @version 1.0.0
 * @author yourname
 * @desc 按年份汇总事件与角色，输出年表面板
 * @host >=0.1.0
 */
export function activate(api) { /* ... */ }
export function deactivate() { /* 释放定时器、监听器、缓存 */ }
```

plugins 表字段	安装时来源
id	界面填写（建议 <code>name@author</code> 形式，需全局唯一）
name / version / author / description	界面填写
code	粘贴或载入的 JS 源码全文
enabled / builtin	用户操作 / 内置插件标记为 1
settings_schema	插件在源码里 <code>export const manifest = { ..., settings: {...} }</code> 声明，宿主载入时回填本列（见 §4.3）
settings	用户在设置表单中填写的值

3 完整示例

3.1 示例一：「世界年鉴」面板插件

按年份聚合时间轴事件与角色出生 / 逝世，点击条目可在宿主中打开对应卡片。

```

/**
 * @plugin world-annals | 世界年鉴
 * 在侧边面板中按年份汇总事件与角色，支持点击跳转卡片。
 */
export function activate(api) {
  const YEAR_KEY = 'year'; // 当前选中年份（存于插件私有存储）

  /** 把时间轴条目与角色按刻度分组 */
  function groupByYear(cards, entries) {
    const years = new Map(); // year -> { events, chars }
    const push = (y, kind, item) => {
      if (!years.has(y)) years.set(y, { events: [], chars: [] });
      years.get(y)[kind].push(item);
    };
    for (const e of entries) push(Math.trunc(e.start_t), 'events', e);
    for (const c of cards) {
      const birth = c.fields && c.fields.birth_t;
      const death = c.fields && c.fields.death_t;
      if (typeof birth === 'number') push(Math.trunc(birth), 'chars', { ...c, kind: '出生' });
      if (typeof death === 'number') push(Math.trunc(death), 'chars', { ...c, kind: '逝世' });
    }
    return [...years.entries()].sort((a, b) => a[0] - b[0]);
  }

  /** 渲染面板：只做 DOM 构建，数据全部来自只读查询 API */
  function render(host, state) {
    const cards = api.cards.query({ type: 'character' });
    const entries = api.cards.query({ type: 'event' });
    const years = groupByYear(cards, entries);
    host.innerHTML = '';
    if (!years.length) { host.textContent = '暂无数据：先创建角色与事件卡片'; return; }

    const selected = Number(api.storage.get(YEAR_KEY) ?? years[0][0]);
    const nav = document.createElement('div');
    nav.className = 'flex flex-wrap gap-1 mb-2';
    for (const [year] of years) {
      const btn = document.createElement('button');
      btn.textContent = String(year);
      btn.className = 'px-2 py-0.5 rounded text-xs ' +
        (year === selected ? 'bg-violet-600 text-white' : 'bg-neutral-700 text-neutral-200');
      btn.onclick = () => { api.storage.set(YEAR_KEY, String(year)); render(host, state); };
      nav.appendChild(btn);
    }
    host.appendChild(nav);

    const bucket = years.find(([y]) => y === selected)?.[1];
    const list = document.createElement('ul');
    list.className = 'space-y-1 text-sm';
    for (const it of [...bucket.events, ...bucket.chars]) {
      const li = document.createElement('li');
      li.className = 'cursor-pointer hover:underline';
      const tag = it.kind ?? '事件';
      li.textContent = `[${tag}] ${it.title}${it.card_id ? ' : ' : '（未绑定卡片）'}`;
      if (it.card_id) li.onclick = () => api.ui.openCard(it.card_id);
      list.appendChild(li);
    }
    host.appendChild(list);
  }

  api.registerPanel({
    id: 'world-annals',
    title: '世界年鉴',
    icon: 'calendar',
    render,
    // (host: HTMLElement, state: { branchId, worldId }) => void
  });
}

```

```
api.registerCommand({
  id: 'world-annals.reset',
  title: '年鉴: 重置年份选择',
  run: () => { api.storage.remove(YEAR_KEY); api.toast('年鉴年份已重置'); },
});
}

export function deactivate() { /* 本插件无定时器与全局监听，无需清理 */ }
```

3.2 示例二：「关系图谱」卡片类型 + 面板插件

注册一个新卡片类型 `graph`（记录一组种子卡片），并提供一个 SVG 一度邻域关系图面板。

```

/**
 * @plugin relation-graph | 关系图谱
 * 新卡片类型 "graph" 保存种子卡片；面板绘制这些种子的一度关系网络。
 */
export function activate(api) {
  // ① 注册新的卡片类型（出现在「新建卡片」的类型选择里）
  api.registerCardType({
    type: 'graph',
    label: '关系图谱',
    icon: 'share-2',
    color: '#38bdf8',
    fields: [
      { key: 'seeds', label: '种子卡片', type: 'link', multiple: true,
        help: '这些卡片及其一度关联会被绘制出来' },
      { key: 'depth_label', label: '图注', type: 'text' },
    ],
  });

  // ② 给已有类型追加一个字段（不改动原有数据，缺省即未填）
  api.registerCardField('character', {
    key: 'graph_group', label: '图谱分组', type: 'text', help: '用于图谱中的聚类着色',
  });

  /** 一度邻域：种子 ∪ 与种子直接相连的卡片 */
  function buildGraph(seedIds) {
    const nodes = new Map(); // id -> { card, seed }
    const edges = [];
    for (const id of seedIds) {
      const card = api.cards.get(id);
      if (card) nodes.set(id, { card, seed: true });
    }
    for (const id of seedIds) {
      for (const rel of api.relations.byCard(id)) {
        const otherId = rel.from_id === id ? rel.to_id : rel.from_id;
        if (!nodes.has(otherId)) {
          const card = api.cards.get(otherId);
          if (card) nodes.set(otherId, { card, seed: false });
        }
        edges.push({ from: rel.from_id, to: rel.to_id, label: rel.label });
      }
    }
    return { nodes: [...nodes.values()], edges };
  }

  /** 环形布局 + SVG 绘制（≤200 节点的轻量实现，不引入图形库） */
  function drawGraph(host, graph) {
    const size = 320, r = size / 2 - 40, cx = size / 2, cy = size / 2;
    const list = graph.nodes.slice(0, 200);
    const pos = new Map(list.map((n, i) => {
      const a = (i / list.length) * Math.PI * 2;
      return [n.card.id, { x: cx + r * Math.cos(a), y: cy + r * Math.sin(a) }]);
    }));
    const esc = (s) => String(s).replace(/[\<>"]/g, (c) =>
      ({ '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;' }[c]));
    const lines = graph.edges
      .filter((e) => pos.has(e.from) && pos.has(e.to))
      .map((e) => {
        const a = pos.get(e.from), b = pos.get(e.to);
        return `<line x1="${a.x}" y1="${a.y}" x2="${b.x}" y2="${b.y}" stroke="#64748b" stroke-width="1"/>`;
      }).join('');
    const dots = list.map((n) => {
      const p = pos.get(n.card.id);
      const fill = n.seed ? (n.card.meta?.color ?? '#a855f7') : '#475569';
      return `<circle cx="${p.x}" cy="${p.y}" r="${n.seed ? 7 : 4}" fill="${fill}" data-card="${n.card.id}"/>` +
        `<text x="${p.x}" y="${p.y + 18}" font-size="9" fill="#e5e7eb" text-anchor="middle">${esc(n.card.title)}</text>`;
    });
  }
}

```

```

    }).join('');
    host.innerHTML = `

```

3.3 示例三：「标记语言导出器」插件

把当前世界观导出为 Markdown / CSV / 自定义 wiki 标记，全部在前端拼字符串，不产生网络请求。

```

/**
 * @plugin markup-export | 标记语言导出器
 * 提供三种导出格式，导出内容由只读查询 API 汇总生成。
 */
export function activate(api) {
  /** 导出为分节 Markdown: 每张卡片一节，关联以列表列出 */
  function toMarkdown(world) {
    const cards = api.cards.query({});
    const byType = new Map();
    for (const c of cards) {
      if (!byType.has(c.type)) byType.set(c.type, []);
      byType.get(c.type).push(c);
    }
    const out = [`# ${world.name} 设定集`, '', `> 导出时间: ${new Date().toISOString()}`, ''];
    for (const [type, list] of byType) {
      out.push(`## ${type} (${list.length})`, '');
      for (const c of list) {
        out.push(`### ${c.title}${c.subtitle ? ` · ${c.subtitle}` : ''}`, '');
        if (c.summary) out.push(c.summary, '');
        for (const rel of api.relations.byCard(c.id)) {
          const other = api.cards.get(rel.from_id === c.id ? rel.to_id : rel.from_id);
          if (other) out.push(`- ${rel.label}: [${other.title}]${rel.note ? ` (${rel.note})` : ''}`);
        }
        if (c.body) out.push('', c.body);
        out.push('');
      }
    }
    return out.join('\n');
  }

  /** 导出为 CSV: 仅核心列，便于在表格软件中筛选 */
  function toCsv() {
    const esc = (v) => `"${String(v ?? '').replace(/"/g, '"')}"`;
    const rows = [['id', 'type', 'title', 'subtitle', 'tags', 'birth_t', 'death_t']];
    for (const c of api.cards.query({})) {
      const tagNames = api.tags.ofCard(c.id).map((t) => t.name).join('|');
      rows.push([c.id, c.type, c.title, c.subtitle ?? '', tagNames,
        c.fields?.birth_t ?? '', c.fields?.death_t ?? '']);
    }
    return rows.map((r) => r.map(esc).join(',')).join('\n');
  }

  /** 导出为自定义 wiki 标记: {{card:标题|显示文本}} */
  function toWikiMarkup(world) {
    const out = [`%% ${world.name} / worldforge-wiki v1`, ''];
    for (const c of api.cards.query({})) {
      out.push(`{{card:${c.id}|${c.title}}}`);
      if (c.body) {
        // 把 [[标题]] 双链转为 wiki 标记
        out.push(c.body.replace(/\[\[([^\]]+)\]\]/g, (_, t) => `{{card:${t}}}`));
      }
      out.push('');
    }
    return out.join('\n');
  }

  const world = () => api.worlds.current();

  api.registerExporter({
    id: 'md-settings', title: '导出 Markdown 设定集', ext: 'md',
    run: () => ({ filename: `${world().name}-设定集.md`, content: toMarkdown(world()) }),
  });
  api.registerExporter({
    id: 'csv-cards', title: '导出卡片表 CSV', ext: 'csv',
    run: () => ({ filename: `${world().name}-卡片.csv`, content: toCsv() }),
  });

```

```
});
api.registerExporter({
  id: 'wiki-markup', title: '导出 Wiki 标记', ext: 'wiki',
  run: () => ({ filename: `${world().name}.wiki`, content: toWikiMarkup(world()) }),
});
}
```

3.4 完整实例：「图库灯箱」——宿主 DOM 契约 + 自建浮层

这一节与实现逐字一致：源码就是仓库里的 `plugins/gallery-lightbox.js`，以内置插件 `builtin.lightbox` 的形式默认启用（同一份文件也能在插件页「从文件载入」）。它演示前三个示例都没覆盖的一类插件：**不新增面板，而是给已有界面加交互**。

宿主与插件之间用 `data-*` 属性约定契约（与正文渲染用的 `data-wiki-id` 是同一思路）：

契约	写在宿主哪里	含义
<code>data-wf-zoom</code>	<code>src/features/cards/CardGallery.tsx</code> 里的图库 <code></code>	标记「这张图可以点开放大」，值为资源 id（只作标记）
<code>data-wf-zoom-group</code>	同上，值为卡片 id	值相同的图片算一组，可在灯箱里左右切换；缺省即单图

插件一侧只做三件事：捕获点击 → 自建浮层（挂到 `document.body`）→ 停用时拆干净。

```
export const manifest = {
  id: 'builtin.lightbox',
  name: '图库灯箱',
  settings: {
    arrows: { type: 'boolean', label: '两侧箭头与计数', default: true },
    loop: { type: 'boolean', label: '首尾循环切换', default: true },
    wheelZoom: { type: 'boolean', label: '滚轮缩放', default: true },
  },
};

export function activate(api) {
  let box = null; // 浮层根节点：非空即表示「已打开」
  let items = []; // 同组图片（按 DOM 顺序 = 图库里的显示顺序）
  let idx = 0;

  /** 读设置：api.settings 是访问器，用户改完设置立即生效 */
  const opt = (key, fallback) => (api.settings[key] === undefined ? fallback : api.settings[key]);

  /** 当前页面里可以放大的图片：隐藏的不算（display:none 的祖先拿不到矩形） */
  function zoomables() {
    const all = [...document.querySelectorAll('img[data-wf-zoom]')];
    return all.filter((img) => !(box && box.contains(img)) && img.getClientRects().length > 0);
  }

  /** 事件委托：点中带标记的图片就打开灯箱 */
  function onClick(event) {
    if (box || event.button !== 0) return;
    const img = event.target.closest('img[data-wf-zoom]');
    if (!img) return;
    event.preventDefault();
    open(img);
  }

  document.addEventListener('click', onClick, true); // 捕获阶段，保证先于 React 处理

  // 宿主停用 / 卸载插件时调用这个清理函数：监听器、浮层、滚动锁定一并还原
  return () => {
    document.removeEventListener('click', onClick, true);
    close();
  };
}
```

要点（每一条都对应一个坑）：

要点	原因
监听挂在 捕获 阶段	宿主的事件处理挂在 React 根节点上，捕获阶段才能保证先跑
全局监听必须成对 <code>removeEventListener</code>	宿主的清理只负责注册项与事件订阅，DOM 监听要插件自己撤（见 §7「界面残留」）
浮层挂 <code>document.body</code> ，不要塞进 React 树	宿主重渲染不会抹掉它，关闭时 <code>remove()</code> 即可
打开时锁 <code>body.style.overflow</code> ，关闭还原	否则滚轮会滚到底下的页面
模态浮层要 <code>stopPropagation</code>	否则宿主 1~9 切模块之类的全局快捷键会被误触
过滤不可见的图片	隐藏面板里的 <code></code> 也在 DOM 里，不过滤就会混进切换列表
显隐用内联 <code>style.display</code> ，少用 <code>hidden</code> 属性	Tailwind 的 <code>flex</code> 等 <code>display</code> 类会盖掉 <code>hidden</code>

为什么不用 `registerPanel`：面板只出现在「插件」模块里，而且 v0.1 的 PluginAPI 没有资源读取接口（插件拿不到图片 URL），宿主已经渲染好的 `<img>` 是唯一现成的图片来源。

代价是插件与宿主 DOM 产生耦合，因此契约在两侧都写明，并由 `scripts/plugin-selftest.mjs` 守着：插件依赖的每个 `data-wf-*` 标记，都必须在宿主源码里真实存在，否则 `npm test` 失败。

4 PluginAPI 参考

以下为 TypeScript 声明风格的契约。`Card` / `Tag` / `Relation` / `CardTypeDef` / `CardFieldDef` 的字段定义见 `docs/数据模型.md` §3、§4。所有查询 API 返回**只读副本**，插件对返回值的修改不影响宿主数据。

```
interface PluginAPI {
  readonly version: string; // 宿主版本，如 "0.1.0"

  // — 卡片类型与字段 —
  registerCardType(def: CardTypeDef): void;
  registerCardField(type: string, field: CardFieldDef): void;

  // — 界面与命令 —
  registerPanel(panel: PanelDef): void;
  registerCommand(cmd: CommandDef): void;

  // — 外观与导出 —
  registerTheme(theme: ThemeDef): void;
  registerExporter(exporter: ExporterDef): void;

  // — 事件 —
  on(event: PluginEvent, handler: (payload: any) => void): () => void;

  // — 只读查询 —
  cards: {
    list(): Card[];
    get(id: string): Card | undefined;
    query(q: CardQuery): Card[];
  };
  tags: { list(): Tag[]; ofCard(cardId: string): Tag[] };
  relations: { byCard(cardId: string): Relation[] };

  // — 宿主上下文与界面 —
  ui: { openCard(cardId: string): void };
  worlds: { current(): { id: string; name: string; meta: any } };

  // — 反馈与存储 —
  toast(message: string, kind?: 'info' | 'success' | 'warn' | 'error'): void;
  storage: {
    get(key: string): string | null;
    set(key: string, value: string): void;
    remove(key: string): void;
    keys(): string[];
  };

  // — 设置（见 §4.3） —
  readonly pluginSettings: Record<string, any>;
  declareSettings(schema: Record<string, SettingFieldDef>): void;
}
```

4.1 各方法签名与用法

方法	参数	返回	说明
registerCardType	{ type, label, icon?, color?, fields: CardFieldDef[] }	void	注册新卡片类型； type 需全局唯一（ 详见
registerCardField	type: string , CardFieldDef	void	为已有类型（含内置）追加字段；同名不迁移数据
registerPanel	{ id, title, icon?, render }	void	注册侧边面板；render(host, state) 由宿主在挂载与数据变更时调用
registerCommand	{ id, title, run, keywords? }	void	注册命令面板指令（Ctrl/Cmd+K）；run
registerTheme	theme: ThemeDef	void	注册配色主题：{ id, name, mode: 'dark' } 覆盖 CSS 变量
registerExporter	exporter: PluginExporterDef	void	注册导出器：{ id, title, ext, run }（ctx 为 { cards, worldName }）

方法	参数	返回	说明
on	event , handler	取消订阅函数	见 §4.2
query.listCards	—	Card[]	当前世界观的全部卡片（按当前分支口径）
query.getCard	id	Card undefined	按 id 取单卡
query.listTags	—	Tag[]	当前世界观的标签表
query.listRelations	—	Relation[]	当前世界观的全部关联；要「某卡的出/入」用 from_id / to_id 过滤
query.listDocs	—	Doc[]	文稿与大纲（用 kind 区分）
query.currentWorldId / query.currentBranchId	—	string null	当前世界观 / 当前分支 id
toast	message , kind?	void	右下角轻提示；错误提示请用 kind='error'
ui.React	—	React 运行时	插件以 Blob 模块载入， 无法 import 'react' ， const { React } = api.ui
settings	—	Record<string, any>	用户在设置表单中填写的值。它是 访问 的， 用户改完立即生效，不需要停用再启用并 default ，请自行兜底
storage	get(key, fallback) / set(key, value)	T / void	插件私有键值存储（落在 plugins.settings 下， 随插件记录持久化）；值可为任意可 JS 序列化

CardFieldDef :

```
interface CardFieldDef {
  key: string; // fields 中的键名, 如 'birth_t'
  label: string; // 表单显示名
  type: 'text' | 'number' | 'tick' | 'select' | 'link' | 'longtext' | 'tags';
  options?: string[]; // type='select' 时的候选项
  multiple?: boolean; // type='link' 时是否允许多选
  unit?: string; // 数值单位提示, 如 '年'
  help?: string; // 字段下的说明文字
  readonly?: boolean; // 只读展示 (由插件计算写入的值时使用)
}

interface SettingFieldDef {
  type: 'text' | 'number' | 'boolean' | 'select' | 'textarea';
  label: string; // 表单显示名
  default: any; // 默认值, 类型须与 type 匹配
  options?: string[]; // type='select' 时的候选项
  help?: string; // 字段下的说明文字
}
```

4.2 事件列表与触发时机

事件	触发时机	payload 结构	典型用途
app:ready	数据初始化完成、store 就绪、插件载入之后	{ worldId, branchId, version }	预热缓存、校验插件设置、提示未配置项
card:save	卡片新增或更新成功提交到 repo 之后	{ card, isNew }	统计字数、自动补字段、写入插件索引
card:delete	卡片删除完成之后	{ cardId, card }	清理插件私有索引、记录日志

```
export function activate(api) {
  const offReady = api.on('app:ready', (s) => api.toast(`已载入世界 ${s.worldId}`));
  const offSave = api.on('card:save', ({ card, isNew }) => {
    if (card.type === 'character' && isNew && !card.fields?.graph_group) {
      api.toast(`新角色「${card.title}」尚未分组`, 'warn');
    }
  });
  // 保存取消订阅函数，供 deactivate 调用（宿主也会在禁用时自动取消）
  api._cleanup = () => { offReady(); offSave(); };
}

export function deactivate() { /* 宿主已统一取消；此处可省略 */ }
```

事件注意事项：① 通知型，不可取消、不可修改 payload；② 回调中不要调用会再次触发同一事件的写 API（会造成循环），需要写数据时请提示用户手动操作；③ 回调抛错会被宿主捕获并计入失败次数，连续 3 次自动禁用该插件。

4.3 设置声明（manifest.settings）

插件在源码里导出 manifest，宿装载入后读取 manifest.settings 并回填 plugins.settings_schema，据此自动渲染设置表单；用户填写的值写回 plugins.settings，插件通过 api.settings 读取。

```
export const manifest = {
  id: 'world-annals',
  name: '世界年鉴',
  settings: {
    yearStart: { type: 'number', label: '年表起始刻度', default: 0 },
    showDead: { type: 'boolean', label: '显示已故角色', default: true },
    palette: { type: 'select', label: '配色', default: 'violet',
      options: [{ value: 'violet', label: '紫罗兰' }, { value: 'emerald', label: '翡翠' }] },
  },
};

export function activate(api) {
  api.registerCommand({
    id: 'annals.export', title: '年鉴：按设置导出',
    run: () => {
      // api.settings 不含 schema 的 default — 必须自己兜底；但它每次读取都是最新值
      const yearStart = Number(api.settings.yearStart ?? 0);
      api.toast(`将从刻度 ${yearStart} 开始`, 'info');
    },
  });
}
```

字段类型与渲染控件的对应（定义见 PluginSettingDef，渲染见 PluginSettingsForm）：

type	控件	值类型
'string'	单行输入	string
'number'	数字输入	number
'boolean'	开关	boolean
'color'	取色器	string
'select'	下拉， 必须 给 options: { value, label }[]	string

另有可选字段：label（表单显示名）、default（默认值）、hint（说明文字）。

生命周期：安装或用内置插件首次载入 → 宿主把 manifest.settings 回填进 plugins.settings_schema（**仅在结构为空时回填一次**，不覆盖已有结构）→ 表单按 settings[key] ?? default 显示 → 用户修改后经 api.updatePluginSettings 写入 plugins.settings → 插件经 api.settings 读到（访问器，改完立即生效，不必重新启用插件）。

5 内置示例插件

插件 id	名称	说明（取自源码里的内置清单）	可卸载
<code>builtin.deepsea</code>	深海配色	两套配色方案，可在设置中选用	可
<code>builtin.stats</code>	写作统计	统计文稿字数与卡片构成；注册面板 + <code>showCards</code> / <code>warnAt</code> 两项设置	可
<code>builtin.namer</code>	随机命名器	一键生成人名地名，可留档	可
<code>builtin.lightbox</code>	图库灯箱	点击图库图片放大预览，可左右切换、键盘翻页、滚轮缩放；源码是仓库里的 <code>plugins/gallery-lightbox.js</code> （见 §3.4）	可

前三个的源码以字符串形式写在 `src/lib/plugin/builtins.ts` 里；第四个例外——它是一份真实存在的独立文件 `plugins/gallery-lightbox.js`，被内置清单用 Vite 的 `?raw` 当文本读进来，所以同一份文件既是内置示例、也能在插件页「从文件载入」，不会两处不同步。

四者都与普通插件走**完全相同**的载入路径，因此是最可靠的编写范本。首次启动时写入数据库，其中 `builtin.deepsea` 与 `builtin.lightbox` 默认启用（内置清单里的 `defaultEnabled`）；四个都可以卸载，卸载后可在插件页用「恢复内置示例」重新装回。

6 调试

手段	用法
控制台日志	插件内直接 <code>console.log</code> ；宿主把日志加上 <code>[plugin:<id>]</code> 前缀输出
状态查看	插件视图列出每个插件的 <code>enabled</code> / <code>builtin</code> / <code>last_error</code> ；异常会写入 <code>last_error</code>
错误提示	主动 <code>api.toast(msg, 'error')</code> ，比静默失败更易定位
局部重载	在插件视图点「禁用 → 启用」即可重新 <code>import()</code> ，无需重启应用（开发时推荐）
断点 / 性能	浏览器端在 DevTools 的 <code>blob</code> ：源下断点（Tauri 端用 WebView 开发者工具）；用 <code>console.time('render')</code> 观测面板 <code>render</code> ，超过 16ms 需缓存或分页
开发循环建议： 在 <code>%TEMP%</code> 或工作区维护一个 <code>plugin.js</code> 源文件 → 每次改动后复制粘贴到插件编辑器 → 禁用 / 启用 → 观察面板与日志。v0.1 不提供文件监听自动重载。	

7 常见问题（FAQ）

问题	原因与解决
启用后没有任何变化 / 事件回调没触发	变化：① 未导出 <code>activate</code> （须为具名导出或 <code>export default { activate }</code> ）；② 注册 <code>id</code> 与已有插件冲突被忽略；③ 面板需手动在侧栏打开。回调：事件名须为 <code>card:save</code> / <code>card:delete</code> / <code>app:ready</code> ，或插件已被自动禁用（见 <code>last_error</code> ）
面板白屏或空白	<code>render</code> 抛错会被宿主捕获；检查是否在数据未就绪时访问了 <code>card.fields.xxx</code> ，请用可选链与默认值
插件界面没有样式	插件里的 Tailwind 类名只有位于被扫描的文件里才会生成 CSS： <code>tailwind.config.js</code> 的 <code>content</code> 含 <code>src/</code> 与 <code>plugins/</code> 。粘贴进来的代码不在扫描范围内，请改用内联 <code>style</code> 或宿主已在用的类名
浮层 / body 上的监听在停用后仍在	宿主的清理只负责注册项与事件订阅；DOM 节点、全局监听、 <code>body.style</code> 要由 <code>activate</code> 返回的清理函数自己还原（§3.4 的写法）
自定义字段不显示	字段定义未注册（ <code>registerCardField</code> 的目标 <code>type</code> 拼写错误）；或该键在定义表中被标记了 <code>readonly</code> 之外的冲突
数值字段变成字符串	<code>fields</code> 是 JSON，写入时请确保是 <code>number</code> （如 <code>Number(v)</code> ），否则时间轴年龄推算会失效
<code>storage</code> 存对象报错 / 读回是 <code>[object Object]</code>	<code>storage</code> 只接受字符串，请自行 <code>JSON.stringify</code> / <code>JSON.parse</code>

问题	原因与解决
主题只改了一部分颜色	<code>tokens</code> 只覆盖已声明的 CSS 变量；未覆盖的仍取默认主题值，请对照宿主主题变量表补全
导出文件名为空或乱码	<code>run()</code> 必须返回 <code>{ filename, content }</code> ；文件名避免 <code>\/:*?"<></code>
禁用插件后界面仍有残留 / 插件报错后自己消失	残留：插件在 <code>activate</code> 之外自行注入了 DOM，请把副作用收敛到 <code>register*</code> ，或从 <code>activate</code> 返回清理函数； 消失：载入或 <code>activate</code> 抛错会让宿主把该插件置为停用（事件回调抛错只打日志、不会停用），修好后重新启用即可

8 发布与分享

v0.1 没有插件市场，分享方式是**单文件 JavaScript**：

方式	步骤	适用场景
粘贴代码	插件视图 → 「新建插件」→ 填 id / 名称 / 版本 / 作者 → 粘贴源码 → 保存并启用	快速试用、短插件
载入文件 / 复制分享	插件视图 → 「载入文件」→ 选择 <code>.js</code> → 确认元信息 → 启用；把 <code>.js</code> 发给他人，对方用同一入口安装	长插件、社区分享

发布前检查清单：

检查项	说明
顶部注释含 id / 名称 / 版本 / 作者 / 说明 / 依赖宿主版本	便于他人判断兼容性
只有一个文件、无外部 <code>import</code>	宿主无依赖解析能力
无网络请求、无遥测	符合本地优先原则（FR-14 / NFR-D1）
所有注册 <code>id</code> 加插件前缀（如 <code>world-annals.*</code> ）	避免与其他插件冲突
从 <code>activate</code> 返回清理函数来释放定时器 / 全局监听	宿主会在停用与卸载时调用，保证无残留
异常路径用 <code>api.toast(..., 'error')</code> 反馈	用户能自行定位问题
README 说明数据落点（用到的表 / 字段 / 私存存储键）	便于用户备份与卸载时决策
声明 <code>settings schema</code> 而非硬编码配置	用户可在界面上调整

同时提醒接收方：**插件无沙箱，安装前请阅读代码**（见 §1.3）。